

Load a string for a specific language or other context

The default [ResourceContext](#) (obtained from [ResourceContext.GetForCurrentView](#)) contains a qualifier value for each qualifier name, representing the default runtime context (in other words, the settings for the current user and machine). Resources Files (.resw) are matched—based on the qualifiers in their names—against the qualifier values in that runtime context.

But there might be times when you want your app to override the system settings and be explicit about the language, scale, or other qualifier value to use when looking for a matching Resources File to load. For example, you might want your users to be able to select an alternative language for tooltips or error messages.

You can do that by constructing a new **ResourceContext** (instead of using the default one), overriding its values, and then using that context object in your string lookups.

C#

```
var resourceContext = new
Windows.ApplicationModel.Resources.Core.ResourceContext(); // not using
ResourceContext.GetForCurrentView
resourceContext.QualifierValues["Language"] = "de-DE";
var resourceMap =
Windows.ApplicationModel.Resources.Core.ResourceManager.Current.MainResource
Map.GetSubtree("Resources");
this.myXAMLTextBlockElement.Text = resourceMap.GetValue("Farewell",
resourceContext).ValueAsString;
```

Using **QualifierValues** as in the code example above works for any qualifier. For the special case of Language, you can alternatively do this instead.

C#

```
resourceContext.Languages = new string[] { "de-DE" };
```

For the same effect at a global level, you *can* override the qualifier values in the default **ResourceContext**. But instead we advise you to call [ResourceContext.SetGlobalQualifierValue](#). You set values one time with a call to **SetGlobalQualifierValue** and then those values are in effect on the default **ResourceContext** each time you use it for lookups.

C#

```
Windows.ApplicationModel.Resources.Core.ResourceContext.SetGlobalQualifierValue("Language", "de-DE");  
var resourceLoader =  
Windows.ApplicationModel.Resources.ResourceLoader.GetForCurrentView();  
this.myXAMLTextBlockElement.Text = resourceLoader.GetString("Farewell");
```

Some qualifiers have a system data provider. So, instead of calling **SetGlobalQualifierValue** you could instead adjust the provider through its own API. For example, this code shows how to set **PrimaryLanguageOverride**.

C#

```
Windows.Globalization.ApplicationLanguages.PrimaryLanguageOverride = "de-DE";
```

Updating strings in response to qualifier value change events

Your running app can respond to changes in system settings that affect the qualifier values in the default **ResourceContext**. Any of these system settings invokes the **MapChanged** event on **ResourceContext.QualifierValues**.

In response to this event, you can reload your strings from the default **ResourceContext**.

C#

```
public MainPage()  
{  
    this.InitializeComponent();  
  
    ...  
  
    // Subscribe to the event that's raised when a qualifier value changes.  
    var qualifierValues =  
Windows.ApplicationModel.Resources.Core.ResourceContext.GetForCurrentView().  
QualifierValues;  
    qualifierValues.MapChanged += new  
Windows.Foundation.Collections.MapChangedEventHandler<string, string>  
(QualifierValues_MapChanged);  
}  
  
private async void QualifierValues_MapChanged(IObservableMap<string, string>  
sender, IMapChangedEventArgs<string> @event)  
{  
    var dispatcher = this.myXAMLTextBlockElement.Dispatcher;
```

```

        if (dispatcher.HasThreadAccess)
        {
            this.RefreshUIText();
        }
        else
        {
            await
dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal, () =>
this.RefreshUIText());
        }
    }

    private void RefreshUIText()
    {
        var resourceLoader =
Windows.ApplicationModel.Resources.ResourceLoader.GetForCurrentView();
        this.myXAMLTextBlockElement.Text = resourceLoader.GetString("Farewell");
    }

```

Load strings from a Class Library or a Windows Runtime Library

The string resources of a referenced Class Library (Universal Windows) or [Windows Runtime Library \(Universal Windows\)](#) are typically added into a subfolder of the package in which they're included during the build process. The resource identifier of such a string usually takes the form *LibraryName/ResourcesFileName/ResourceIdentifier*.

A library can get a ResourceLoader for its own resources. For example, the following code illustrates how either a library or an app that references it can get a ResourceLoader for the library's string resources.

C#

```

var resourceLoader =
Windows.ApplicationModel.Resources.ResourceLoader.GetForCurrentView("Contoso
Control/Resources");
this.myXAMLTextBlockElement.Text =
resourceLoader.GetString("exampleResourceName");

```

For a Windows Runtime Library (Universal Windows), if the default namespace is segmented (it contains "." characters), then use dots in the resource map name.

C#

```

var resourceLoader =
Windows.ApplicationModel.Resources.ResourceLoader.GetForCurrentView("Contoso
.Control/Resources");

```

You don't need to do that for a Class Library (Universal Windows). If in doubt, you can specify [MakePri.exe command line options](#) to dump your component or library's PRI file. Each resource's `uri` is shown in the dumped file.

XML

```
<NamedResource name="exampleResourceName" uri="ms-  
resource://Contoso.Control/Contoso.Control/ReswFileName/exampleResourceName"  
>...
```

Loading strings from other packages

The resources for an app package are managed and accessed through the package's own top-level [ResourceMap](#) that's accessible from the current [ResourceManager](#). Within each package, various components can have their own ResourceMap subtrees, which you can access via [ResourceMap.GetSubtree](#).

A framework package can access its own resources with an absolute resource identifier URI. Also see [URI schemes](#).

Loading strings in unpackaged applications

As of Windows Version 1903 (May 2019 Update), unpackaged applications can also leverage the Resource Management System.

Just create your UWP user controls/libraries and [store any strings in a resources file](#). You can then [refer to a string resource identifier from XAML](#), [refer to a string resource identifier from code](#), or [load strings from a Class Library or a Windows Runtime Library](#).

To use resources in unpackaged applications, you should do a few things:

1. Use [GetForViewIndependentUse](#) instead of [GetForCurrentView](#) when resolving resources from code as there is no *current view* in unpackaged scenarios. The following exception occurs if you call [GetForCurrentView](#) in unpackaged scenarios: *Resource Contexts may not be created on threads that do not have a CoreWindow*.
2. Use [MakePri.exe](#) to manually generate your app's resources.pri file.

- Run `makepri new /pr <PROJECTROOT> /cf <PRICONFIG> /of resources.pri`
- The <PRICONFIG> must omit the "<packaging>" section so that all resources are bundled in a single resources.pri file. If using the default [MakePri.exe](#)

[configuration file](#) created by [createconfig](#), you need to delete the "<packaging>" section manually after it is created.

- The <PRICONFIG> must contain all relevant indexers required to merge all resources in your project into a single resources.pri file. The default [MakePri.exe configuration file](#) created by [createconfig](#) includes all indexers.
- If you don't use the default config, make sure the PRI indexer is enabled (review the default config for how to do this) to merge PRIs found from UWP project references, NuGet references, and so on, that are located within the project root.

ⓘ Note

By omitting `/IndexName`, and by the project not having an app manifest, the `IndexName/root` namespace of the PRI file is automatically set to *Application*, which the runtime understands for unpackaged apps (this removes the previous hard dependency on package ID). When specifying resource URIs, `ms-resource:///` references that omit the root namespace infer *Application* as the root namespace for unpackaged apps (or you can specify *Application* explicitly as in `ms-resource://Application/`).

3. Copy the PRI file to the build output directory of the .exe

4. Run the .exe

ⓘ Note

The Resource Management System uses the system display language rather than the user preferred language list when resolving resources based on language in unpackaged apps. The user preferred language list is only used for UWP apps.

ⓘ Important

You must manually rebuild PRI files whenever resources are modified. We recommend using a post-build script that handles the [MakePri.exe](#) command and copies the resources.pri output to the .exe directory.

Important APIs

- [ApplicationModel.Resources.ResourceLoader](#)
- [ResourceContext.SetGlobalQualifierValue](#)
- [MapChanged](#)

Related topics

- [Porting XAML and UI](#)
- [x:Uid directive](#)
- [attached properties](#)
- [Localizable manifest items](#)
- [BCP-47 language tag](#) [↗](#)
- [Tailor your resources for language, scale, and other qualifiers](#)
- [How to load string resources](#)

Load images and assets tailored for scale, theme, high contrast, and more

Article • 08/16/2023

Your app can load image resource files (or other asset files) tailored for [display scale factor](#), theme, high contrast, and other runtime contexts. These images can be referenced from imperative code or from XAML markup, for example as the **Source** property of an **Image**. They can also appear in your app package manifest source file (the `Package.appxmanifest` file)—for example, as the value for App Icon on the Visual Assets tab of the Visual Studio Manifest Designer—or on your tiles and toasts. By using qualifiers in your images' file names, and optionally dynamically loading them with the help of a [ResourceContext](#), you can cause the most appropriate image file to be loaded that best matches the user's runtime settings for display scale, theme, high contrast, language, and other contexts.

An image resource is contained in an image resource file. You can also think of the image as an asset, and the file that contains it as an asset file; and you can find these kinds of resource files in your project's `\Assets` folder. For background on how to use qualifiers in the names of your image resource files, see [Tailor your resources for language, scale, and other qualifiers](#).

Some common qualifiers for images are [scale](#), [theme](#), [contrast](#), and [targetsize](#).

Qualify an image resource for scale, theme, and contrast

The default value for the `scale` qualifier is `scale-100`. So, these two variants are equivalent (they both provide an image at scale 100, or scale factor 1).

```
\Assets\Images\logo.png  
\Assets\Images\logo.scale-100.png
```

You can use qualifiers in folder names instead of file names. That would be a better strategy if you have several asset files per qualifier. For purposes of illustration, these two variants are equivalent to the two above.

```
\Assets\Images\logo.png  
\Assets\Images\scale-100\logo.png
```

Next is an example of how you can provide variants of an image resource—named `/Assets/Images/logo.png`—for different settings of display scale, theme, and high contrast. This example uses folder naming.

```
\Assets\Images\contrast-standard\theme-dark  
    \scale-100\logo.png  
    \scale-200\logo.png  
\Assets\Images\contrast-standard\theme-light  
    \scale-100\logo.png  
    \scale-200\logo.png  
\Assets\Images\contrast-high  
    \scale-100\logo.png  
    \scale-200\logo.png
```

Reference an image or other asset from XAML markup and code

The name—or identifier—of an image resource is its path and file name with any and all qualifiers removed. If you name folders and/or files as in any of the examples in the previous section, then you have a single image resource and its name (as an absolute path) is `/Assets/Images/logo.png`. Here's how you use that name in XAML markup.

XAML

```
<Image x:Name="myXAMLImageElement" Source="ms-appx:///Assets/Images/logo.png"/>
```

Notice that you use the `ms-appx` URI scheme because you're referring to a file that comes from your app's package. See [URI schemes](#). And here's how you refer to the same image resource in imperative code.

C#

```
this.myXAMLImageElement.Source = new BitmapImage(new Uri("ms-appx:///Assets/Images/logo.png"));
```

You can use `ms-appx` to load any arbitrary file from your app package.

C#

```
var uri = new System.Uri("ms-appx:///Assets/anyAsset.ext");  
var storagefile = await  
Windows.Storage.StorageFile.GetFileFromApplicationUriAsync(uri);
```

The `ms-appx-web` scheme accesses the same files as `ms-appx`, but in the web compartment.

XAML

```
<WebView x:Name="myXAMLWebViewElement" Source="ms-appx-  
web:///Pages/default.html"/>
```

C#

```
this.myXAMLWebViewElement.Source = new Uri("ms-appx-  
web:///Pages/default.html");
```

For any of the scenarios shown in these examples, use the [Uri constructor](#) overload that infers the [UriKind](#). Specify a valid absolute URI including the scheme and authority, or just let the authority default to the app's package as in the example above.

Notice how in these example URIs the scheme ("`ms-appx`" or "`ms-appx-web`") is followed by "`://`" which is followed by an absolute path. In an absolute path, the leading "/" causes the path to be interpreted from the root of the package.

ⓘ Note

The `ms-resource` (for **string resources**) and `ms-appx(-web)` (for images and other assets) URI schemes perform automatic qualifier matching to find the resource that's most appropriate for the current context. The `ms-appdata` URI scheme (which is used to load app data) does not perform any such automatic matching, but you can respond to the contents of [ResourceContext.QualifierValues](#) and explicitly load the appropriate assets from app data using their full physical file name in the URI. For info about app data, see [Store and retrieve settings and other app data](#). Web URI schemes (for example, `http`, `https`, and `ftp`) do not perform automatic matching, either. For info about what to do in that case, see [Hosting and loading images in the cloud](#).

Absolute paths are a good choice if your image files remain where they are in the project structure. If you want to be able to move an image file, but you're careful that it

remains in the same location relative to its referencing XAML markup file, then instead of an absolute path you might want to use a path that's relative to the containing markup file. If you do that, then you needn't use a URI scheme. You will still benefit from automatic qualifier matching in this case, but only because you are using the relative path in XAML markup.

XAML

```
<Image Source="Assets/Images/logo.png"/>
```

Also see [Tile and toast support for language, scale, and high contrast](#).

Qualify an image resource for targetsize

You can use the `scale` and `targetsize` qualifiers on different variants of the same image resource; but you can't use them both on a single variant of a resource. Also, you need to define at least one variant without a `TargetSize` qualifier. That variant must either define a value for `scale`, or let it default to `scale-100`. So, these two variants of the `/Assets/Square44x44Logo.png` resource are valid.

```
\Assets\Square44x44Logo.scale-200.png  
\Assets\Square44x44Logo.targetsize-24.png
```

And these two variants are valid.

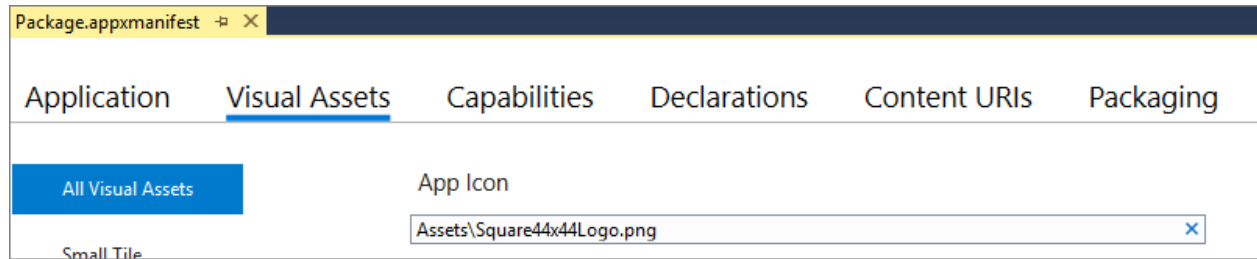
```
\Assets\Square44x44Logo.png // defaults to scale-100  
\Assets\Square44x44Logo.targetsize-24.png
```

But this variant is not valid.

```
\Assets\Square44x44Logo.scale-200_targetsize-24.png
```

Refer to an image file from your app package manifest

If you name folders and/or files as in either of the two valid examples in the previous section, then you have a single app icon image resource and its name (as a relative path) is `Assets\Square44x44Logo.png`. In your app package manifest, simply refer to the resource by name. There's no need to use any URI scheme.



That's all you need to do, and the OS will perform automatic qualifier matching to find the resource that's most appropriate for the current context. For a list of all items in the app package manifest that you can localize or otherwise qualify in this way, see [Localizable manifest items](#).

Qualify an image resource for layoutdirection

See [Mirroring images](#).

Load an image for a specific language or other context

For more info about the value proposition of localizing your app, see [Globalization and localization](#).

The default [ResourceContext](#) (obtained from [ResourceContext.GetForCurrentView](#)) contains a qualifier value for each qualifier name, representing the default runtime context (in other words, the settings for the current user and machine). Image files are matched—based on the qualifiers in their names—against the qualifier values in that runtime context.

But there might be times when you want your app to override the system settings and be explicit about the language, scale, or other qualifier value to use when looking for a matching image to load. For example, you might want to control exactly when and which high contrast images are loaded.

You can do that by constructing a new **ResourceContext** (instead of using the default one), overriding its values, and then using that context object in your image lookups.

```

var resourceContext = new
Windows.ApplicationModel.Resources.Core.ResourceContext(); // not using
ResourceContext.GetForCurrentView
resourceContext.QualifierValues["Contrast"] = "high";
var namedResource =
Windows.ApplicationModel.Resources.Core.ResourceManager.Current.MainResource
Map["@Files/Assets/Logo.png"];
var resourceCandidate = namedResource.Resolve(resourceContext);
var imageFileStream =
resourceCandidate.GetValueAsStreamAsync().GetResults();
var bitmapImage = new Windows.UI.Xaml.Media.Imaging.BitmapImage();
bitmapImage.SetSourceAsync(imageFileStream);
this.myXAMLImageElement.Source = bitmapImage;

```

To achieve same effect at a global level, you *can* override the qualifier values in the default **ResourceContext**. But instead we advise you to call **ResourceContext.SetGlobalQualifierValue**. You set values one time with a call to **SetGlobalQualifierValue** and then those values are in effect on the default **ResourceContext** each time you use it for lookups. By default, the **ResourceManager** class uses the default **ResourceContext**.

C#

```

Windows.ApplicationModel.Resources.Core.ResourceContext.SetGlobalQualifierVa
lue("Contrast", "high");
var namedResource =
Windows.ApplicationModel.Resources.Core.ResourceManager.Current.MainResource
Map["@Files/Assets/Logo.png"];
this.myXAMLImageElement.Source = new
Windows.UI.Xaml.Media.Imaging.BitmapImage(namedResource.Uri);

```

Updating images in response to qualifier value change events

Your running app can respond to changes in system settings that affect the qualifier values in the default resource context. Any of these system settings invokes the **MapChanged** event on **ResourceContext.QualifierValues**.

In response to this event, you can reload your images with the help of the default **ResourceContext**, which **ResourceManager** uses by default.

C#

```

public MainPage()
{

```

```

        this.InitializeComponent();

        ...

        // Subscribe to the event that's raised when a qualifier value changes.
        var qualifierValues =
Windows.ApplicationModel.Resources.Core.ResourceContext.GetForCurrentView().
QualifierValues;
        qualifierValues.MapChanged += new
Windows.Foundation.Collections.MapChangedEventHandler<string, string>
(QualifierValues_MapChanged);
    }

    private async void QualifierValues_MapChanged(IObservableMap<string, string>
sender, IMapChangedEventArgs<string> @event)
    {
        var dispatcher = this.myImageXAMLElement.Dispatcher;
        if (dispatcher.HasThreadAccess)
        {
            this.RefreshUIImages();
        }
        else
        {
            await
dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal, () =>
this.RefreshUIImages());
        }
    }

    private void RefreshUIImages()
    {
        var namedResource =
Windows.ApplicationModel.Resources.Core.ResourceManager.Current.MainResource
Map["@Files/Assets/Logo.png"];
        this.myImageXAMLElement.Source = new
Windows.UI.Xaml.Media.Imaging.BitmapImage(namedResource.Uri);
    }

```

Important APIs

- [ResourceContext](#)
- [ResourceContext.SetGlobalQualifierValue](#)
- [MapChanged](#)

Related topics

- [Tailor your resources for language, scale, and other qualifiers](#)
- [Localize strings in your UI and app package manifest](#)
- [Store and retrieve settings and other app data](#)

- Tile and toast support for language, scale, and high contrast
- Localizable manifest items
- Mirroring images
- Globalization and localization

URI schemes

Article • 10/20/2022

There are several URI (Uniform Resource Identifier) schemes that you can use to refer to files that come from your app's package, your app's data folders, or the cloud. You can also use a URI scheme to refer to strings loaded from your app's Resources Files (.resw). You can use these URI schemes in your code, in your XAML markup, in your app package manifest, or in your tile and toast notification templates.

Common features of the URI schemes

All of the schemes described in this topic follow typical URI scheme rules for normalization and resource retrieval. See [RFC 3986](#) for the generic syntax of a URI.

All of the URI schemes define the hierarchical part per [RFC 3986](#) as the authority and path components of the URI.

syntax

```
URI          = scheme ":" hier-part [ "?" query ] [ "#" fragment ]
hier-part    = "://" authority path-abempty
              / path-absolute
              / path-rootless
              / path-empty
```

What this means is that there are essentially three components to a URI. Immediately following the two forward slashes of the URI *scheme* is a component (which can be empty) called the *authority*. And immediately following that is the *path*. Taking the URI `http://www.contoso.com/welcome.png` as an example, the scheme is "`http://`", the authority is "`www.contoso.com`", and the path is "`/welcome.png`". Another example is the URI `ms-appx:///logo.png`, where the authority components is empty and takes a default value.

The fragment component is ignored by the scheme-specific processing of the URIs mentioned in this topic. During resource retrieval and comparison, the fragment component has no bearing. However, layers above specific implementation may interpret the fragment to retrieve a secondary resource.

Comparison occurs byte for byte after normalization of all IRI components.

Case-insensitivity and normalization

All the URI schemes described in this topic follow typical URI rules (RFC 3986) for normalization and resource retrieval for schemes. The normalized form of these URIs maintains case and percent-decodes RFC 3986 unreserved characters.

For all the URI schemes described in this topic, *scheme*, *authority*, and *path* are either case-insensitive by standard, or else are processed by the system in a case-insensitive way. **Note** The only exception to that rule is the *authority* of `ms-resource`, which is case-sensitive.

ms-appx and ms-appx-web

Use the `ms-appx` or the `ms-appx-web` URI scheme to refer to a file that comes from your app's package (see [Packaging apps](#)). Files in your app package are typically static images, data, code, and layout files. The `ms-appx-web` scheme accesses the same files as `ms-appx`, but in the web compartment. For examples and more info, see [Reference an image or other asset from XAML markup and code](#).

Scheme name (ms-appx and ms-appx-web)

The URI scheme name is the string "ms-appx" or "ms-appx-web".

XML

```
ms-appx://
```

XML

```
ms-appx-web://
```

Authority (ms-appx and ms-appx-web)

The authority is the package identity name that is defined in the package manifest. It is therefore limited in both the URI and IRI (Internationalized resource identifier) form to the set of characters allowed in a package identity name. The package name must be the name of one of the packages in the current running app's package dependency graph.

XML

```
ms-appx://Contoso.MyApp/
```

```
ms-appx-web://Contoso.MyApp/
```

If any other character appears in the authority, then retrieval and comparison fail. The default value for the authority is the currently running app's package.

XML

```
ms-appx:///
ms-appx-web:///
```

User info and port (ms-appx and ms-appx-web)

The `ms-appx` scheme, unlike other popular schemes, does not define a user info or port component. Since "@" and ":" are not allowed as valid authority values, lookup will fail if they are included. Each of the following fails.

XML

```
ms-appx://john@contoso.myapp/default.html
ms-appx://john:password@contoso.myapp/default.html
ms-appx://contoso.myapp:8080/default.html
ms-appx://john:password@contoso.myapp:8080/default.html
```

Path (ms-appx and ms-appx-web)

The path component matches the generic RFC 3986 syntax and supports non-ASCII characters in IRIs. The path component defines the logical or physical file path of a file. That file is in a folder associated with the installed location of the app package, for the app specified by the authority.

If the path refers to a physical path and file name then that physical file asset is retrieved. But if no such physical file is found then the actual resource returned during retrieval is determined by using content negotiation at runtime. This determination is based on app, OS, and user settings such as language, display scale factor, theme, high contrast, and other runtime contexts. For example, a combination of the app's languages, the system's display settings, and the user's high contrast settings may be taken into account when determining the actual resource value to be retrieved.

XML

```
ms-appx:///images/logo.png
```

The URI above may actually retrieve a file within the current app's package with the following physical file name.

```
\Images\fr-FR\logo.scale-100_contrast-white.png
```

You could of course also retrieve that same physical file by referring to it directly by its full name.

XAML

```
<Image Source="ms-appx:///images/fr-FR/logo.scale-100_contrast-white.png"/>
```

The path component of `ms-appx(-web)` is, like generic URIs, case sensitive. However, when the underlying file system by which the resource is accessed is case insensitive, such as for NTFS, the retrieval of the resource is done case-insensitively.

The normalized form of the URI maintains case, and percent-decodes (a "%" symbol followed by the two-digit hexadecimal representation) RFC 3986 unreserved characters. The characters "?", "#", "/", "*", and '"' (the double-quote character) must be percent-encoded in a path to represent data such as file or folder names. All percent-encoded characters are decoded before retrieval. Thus, to retrieve a file named Hello#World.html, use this URI.

XML

```
ms-appx:///Hello%23World.html
```

Query (ms-appx and ms-appx-web)

Query parameters are ignored during retrieval of resources. The normalized form of query parameters maintains case. Query parameters are not ignored during comparison.

ms-appdata

Use the `ms-appdata` URI scheme to refer to files that come from the app's local, roaming, and temporary data folders. For more info about these app data folders, see [Store and retrieve settings and other app data](#).

The `ms-appdata` URI scheme does not perform the runtime content negotiation that [ms-appx](#) and [ms-appx-web](#) do. But you can respond to the contents of

[ResourceContext.QualifierValues](#) and load the appropriate assets from app data using their full physical file name in the URI.

Scheme name (ms-appdata)

The URI scheme name is the string "ms-appdata".

XML

```
ms-appdata://
```

Authority (ms-appdata)

The authority is the package identity name that is defined in the package manifest. It is therefore limited in both the URI and IRI (Internationalized resource identifier) form to the set of characters allowed in a package identity name. The package name must be the name of the current running app's package.

XML

```
ms-appdata://Contoso.MyApp/
```

If any other character appears in the authority, then retrieval and comparison fail. The default value for the authority is the currently running app's package.

XML

```
ms-appdata:///
```

User info and port (ms-appdata)

The `ms-appdata` scheme, unlike other popular schemes, does not define a user info or port component. Since "@" and ":" are not allowed as valid authority values, lookup will fail if they are included. Each of the following fails.

XML

```
ms-appdata://john@contoso.myapp/local/data.xml  
ms-appdata://john:password@contoso.myapp/local/data.xml  
ms-appdata://contoso.myapp:8080/local/data.xml  
ms-appdata://john:password@contoso.myapp:8080/local/data.xml
```

Path (ms-appdata)

The path component matches the generic RFC 3986 syntax and supports non-ASCII characters in IRIs. Within the [Windows.Storage.ApplicationData](#) location are three reserved folders for local, roaming, and temporary state storage. The `ms-appdata` scheme allows access to files and folders in those locations. The first segment of the path component must specify the particular folder in the following fashion. Thus the "path-empty" form of "hier-part" is not legal.

Local folder.

XML

```
ms-appdata:///local/
```

Temporary folder.

XML

```
ms-appdata:///temp/
```

Roaming folder.

XML

```
ms-appdata:///roaming/
```

The path component of `ms-appdata` is, like generic URIs, case sensitive. However, when the underlying file system by which the resource is accessed is case insensitive, such as for NTFS, the retrieval of the resource is done case-insensitively.

The normalized form of the URI maintains case, and percent-decodes (a "%" symbol followed by the two-digit hexadecimal representation) RFC 3986 unreserved characters. The characters "?", "#", "/", "*", and "" (the double-quote character) must be percent-encoded in a path to represent data such as file or folder names. All percent-encoded characters are decoded before retrieval. Thus, to retrieve a local file named `Hello#World.html`, use this URI.

XML

```
ms-appdata:///local/Hello%23World.html
```

Retrieval of the resource, and identification of the top level path segment, are handled after normalization of dots ("../b/c"). Therefore, URIs cannot dot themselves out of one of the reserved folders. Thus, the following URI is not allowed.

XML

```
ms-appdata:///local/../../hello/logo.png
```

But this URI is allowed (albeit redundant).

XML

```
ms-appdata:///local/../../roaming/logo.png
```

Query (ms-appdata)

Query parameters are ignored during retrieval of resources. The normalized form of query parameters maintains case. Query parameters are not ignored during comparison.

ms-resource

Use the `ms-resource` URI scheme to refer to strings loaded from your app's Resources Files (.resw). For examples and more info about Resources Files, see [Localize strings in your UI and app package manifest](#).

Scheme name (ms-resource)

The URI scheme name is the string "ms-resource".

XML

```
ms-resource://
```

Authority (ms-resource)

The authority is the top-level resource map defined in the Package Resource Index (PRI), which typically corresponds to the package identity name that is defined in the package manifest. See [Packaging apps](#). It is therefore limited in both the URI and IRI (Internationalized resource identifier) form to the set of characters allowed in a package

identity name. The package name must be the name of one of the packages in the current running app's package dependency graph.

XML

```
ms-resource://Contoso.MyApp/  
ms-resource://Microsoft.WinJS.1.0/
```

If any other character appears in the authority, then retrieval and comparison fail. The default value for the authority is the case-sensitive package name of the currently running app.

XML

```
ms-resource:///
```

The authority is case sensitive, and the normalized form maintains its case. Lookup of a resource, however, happens case-insensitively.

User info and port (ms-resource)

The `ms-resource` scheme, unlike other popular schemes, does not define a user info or port component. Since "@" and ":" are not allowed as valid authority values, lookup will fail if they are included. Each of the following fails.

XML

```
ms-resource://john@contoso.myapp/Resources/String1  
ms-resource://john:password@contoso.myapp/Resources/String1  
ms-resource://contoso.myapp:8080/Resources/String1  
ms-resource://john:password@contoso.myapp:8080/Resources/String1
```

Path (ms-resource)

The path identifies the hierarchical location of the [ResourceMap](#) subtree (see [Resource Management System](#)) and the [NamedResource](#) within it. Typically, this corresponds to the filename (excluding extension) of a Resources File (.resw) and the identifier of a string resource within it.

For examples and more info, see [Localize strings in your UI and app package manifest](#) and [Tile and toast notification support for language, scale, and high contrast](#).

The path component of `ms-resource` is, like generic URIs, case sensitive. However, the underlying retrieval does a [CompareStringOrdinal](#) with *ignoreCase* set to `true`.

The normalized form of the URI maintains case, and percent-decodes (a "%" symbol followed by the two-digit hexadecimal representation) RFC 3986 unreserved characters. The characters "?", "#", "/", "*", and "" (the double-quote character) must be percent-encoded in a path to represent data such as file or folder names. All percent-encoded characters are decoded before retrieval. Thus, to retrieve a string resource from a Resources File named `Hello#World.resw`, use this URI.

XML

```
ms-resource:///Hello%23World/String1
```

Query (ms-resource)

Query parameters are ignored during retrieval of resources. The normalized form of query parameters maintains case. Query parameters are not ignored during comparison. Query parameters are compared case-sensitively.

Developers of particular components layered above this URI parsing may choose to use the query parameters as they see fit.

Related topics

- [Uniform Resource Identifier \(URI\): Generic Syntax](#) 
- [Packaging apps](#)
- [Reference an image or other asset from XAML markup and code](#)
- [Store and retrieve settings and other app data](#)
- [Localize strings in your UI and app package manifest](#)
- [Resource Management System](#)
- [Tile and toast notification support for language, scale, and high contrast](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Specify the default resources that your app uses

Article • 10/20/2022

If your app doesn't have resources that match the particular settings of a customer device, then the app's default resources are used. This topic explains how to specify what those default resources are.

When a customer installs your app from the Microsoft Store, settings on the customer's device are matched against the app's available resources. This matching is done so that only the appropriate resources need to be downloaded and installed for that user. For example, the most appropriate strings and images for the user's language preferences, and the device's resolution and DPI settings, are used. For example, `200` is the default value for `scale`, but you can override that default if you wish.

Even for resources that don't go into their own resource packs (such as images tailored for high contrast settings), you can specify what default resources the app should use at run time if a resource that matches the user's settings can't be found. For example, `standard` is the default value for `contrast`, but you can override that default if you wish.

These defaults are specified in the form of default resource qualifier values. For an explanation of what resource qualifiers are, their use, and purpose, see [Tailor your resources for language, scale, high contrast, and other qualifiers](#).

You can configure what these defaults are in one of two ways. You can either add a configuration file to your project, or you can edit your project file directly. Use whichever of these options you're most comfortable with, or whichever works best with your build system.

Option 1. Use `priconfig.default.xml` to specify default qualifier values

1. In Visual Studio, add a new item to your project. Choose XML File, and name the file `priconfig.default.xml`.
2. In Solution Explorer, select `priconfig.default.xml` and check the Properties window. The file's Build Action should be set to None, and Copy to Output Directory should be set to Do not copy.
3. Replace the contents of the file with this XML.

XML

```
<default>
  <qualifier name="Language" value="LANGUAGE-TAG(S)" />
  <qualifier name="Contrast" value="standard" />
  <qualifier name="Scale" value="200" />
  <qualifier name="HomeRegion" value="001" />
  <qualifier name="TargetSize" value="256" />
  <qualifier name="LayoutDirection" value="LTR" />
  <qualifier name="DXFeatureLevel" value="DX9" />
  <qualifier name="Configuration" value="" />
  <qualifier name="AlternateForm" value="" />
</default>
```

Note The value `LANGUAGE-TAG(S)` needs to be synchronized with your app's default language. If that's a single [BCP-47 language tag](#), then your app's default language needs to be the same tag. If it's a comma-separated list of language tags, then your app's default language needs to be the first tag in the list. You set your app's default language in the **Default language** field on the **Application** tab in your app package manifest source file (`Package.appxmanifest`).

- Each `<qualifier>` element tells Visual Studio what value to use as the default for each qualifier name. With the file contents you have so far, you haven't actually changed Visual Studio's behavior. In other words, Visual Studio *already behaved as if* this file were present with these contents, because these are the default defaults. So to override a default with your own default value, you'll have to change a value in the file. Here's an example of how the file might look if you'd edited the first three values.

XML

```
<default>
  <qualifier name="Language" value="de-DE" />
  <qualifier name="Contrast" value="black" />
  <qualifier name="Scale" value="400" />
  <qualifier name="HomeRegion" value="001" />
  <qualifier name="TargetSize" value="256" />
  <qualifier name="LayoutDirection" value="LTR" />
  <qualifier name="DXFeatureLevel" value="DX9" />
  <qualifier name="Configuration" value="" />
  <qualifier name="AlternateForm" value="" />
</default>
```

- Save and close the file and rebuild your project.

To confirm that your overridden defaults are being taken into account, look for the file `<ProjectFolder>\obj\<ReleaseConfiguration folder>\priconfig.xml` and confirm that its contents match your overrides. If they do, then you have successfully configured the qualifier values of the resources that your app will use by default. If a match for the user's settings is not found, then resources will be used whose folder or file names contain the default qualifier values that you've set here.

How does this work?

Behind the scenes, Visual Studio launches a tool named `MakePri.exe` to generate a file known as a Package Resource Index (PRI), which describes all of your app's resources, including indicating which are the default resources. For details about this tool, see [Compile resources manually with MakePri.exe](#). Visual Studio passes a configuration file to `MakePri.exe`. The contents of your `priconfig.default.xml` file are used as the `<default>` element of that configuration file, which is the part that specifies the set of qualifier values that are considered to be default. So, adding and editing `priconfig.default.xml` ultimately influences the contents of the Package Resource Index file that Visual Studio generates for your app and includes in its app package.

Note Any time you change the value of the `<qualifier name="Language" ... />` element, you need to synchronize that change with your app's default language. This is so that the language resources indexed in your app's PRI file match your app's manifest default language. The value in the `<qualifier name="Language" ... />` element overrides the value in the manifest with respect to the contents of `<ProjectFolder>\obj\<ReleaseConfiguration folder>\priconfig.xml`, but that file and your app's manifest should match.

Using a different file name than `priconfig.default.xml`

If you name your file `priconfig.default.xml`, then Visual Studio will recognize it and use it automatically. If you give it a different name, then you'll need to let Visual Studio know. In your project file, between the opening and closing tags of the first `<PropertyGroup>` element, add this XML.

XML

```
<AppxPriConfigXmlDefaultSnippetPath>FILE-PATH-AND-NAME</AppxPriConfigXmlDefaultSnippetPath>
```

Replace `FILE-PATH-AND-NAME` with the path to, and name of, your file.

Option 2. Use your project file to specify default qualifier values

This is an alternative to Option 1. Once you understand how Option 1 works, you can choose to do Option 2 instead, if that suits your development and/or build workflow better.

In your project file, between the opening and closing tags of the first `<PropertyGroup>` element, add this XML.

XML

```
<AppxDefaultResourceQualifiers>Language=LANGUAGE-  
TAG(S)|Contrast=standard|Scale=200|HomeRegion=001|TargetSize=256|LayoutDirec  
tion=LTR|DXFeatureLevel=DX9|Configuration=|AlternateForm=  
</AppxDefaultResourceQualifiers>
```

Here's an example of how that might look after you've edited the first three values.

XML

```
<AppxDefaultResourceQualifiers>Language=de-  
DE|Contrast=black|Scale=400|HomeRegion=001|TargetSize=256|LayoutDirection=LT  
R|DXFeatureLevel=DX9|Configuration=|AlternateForm=  
</AppxDefaultResourceQualifiers>
```

Save and close, and rebuild your project.

Note Any time you change the `Language=` value, you need to synchronize that change with your app's default language in the manifest designer (by opening `Package.appxmanifest`).

Related topics

- [Tailor your resources for language, scale, high contrast, and other qualifiers](#)
- [BCP-47 language tag](#) ↗
- [Compile resources manually with MakePri.exe](#)

Build resources into your app package instead of into a resource pack

Article • 04/27/2023

Some kinds of apps (multilingual dictionaries, translation tools, etc.) need to override the default behavior of an app bundle and build resources into the app package instead of having them in separate resource packages (or resource packs). This topic explains how to do that.

By default, when you build an [app bundle \(.appxbundle\)](#), only your default resources for language, scale, and DirectX feature level are built into the app package. Your translated resources—and your resources tailored for non-default scales and/or DirectX feature levels—are built into resource packages, and they are only downloaded onto devices that need them. If a customer is buying your app from the Microsoft Store using a device with a language preference set to Spanish, then only your app plus the Spanish resource package are downloaded and installed. If that same user later changes their language preference to French in **Settings**, then your app's French resource package is downloaded and installed. Similar things happen with your resources qualified for scale and for DirectX feature level. For the majority of apps, this behavior constitutes a valuable efficiency, and it's what you and the customer *want* to happen.

But if your app allows the user to change the language on the fly from within the app (instead of via **Settings**), then that default behavior is not appropriate. You actually want all of your language resources to be unconditionally downloaded and installed along with the app one time, and then remain on the device. You want to build all of those resources into your app package instead of into separate resource packages.

Note Including resources in an app package essentially increases the size of the app. That's why it's only worth doing if the nature of the app demands it. If not, then you don't need to do anything except build a regular app bundle as usual.

You can configure Visual Studio to build resources into your app package in one of two ways. You can either add a configuration file to your project or you can edit your project file directly. Use whichever of these options you're most comfortable with or whichever works best with your build system.

Option 1. Use `priconfig.packaging.xml` to build resources into your app package

1. In Visual Studio, add a new item to your project. Choose XML File, and name the file `priconfig.packaging.xml`.
2. In Solution Explorer, select `priconfig.packaging.xml` and check the Properties window. The file's Build Action should be set to None, and Copy to Output Directory should be set to Do not copy.
3. Replace the contents of the file with this XML.

XML

```
<packaging>
  <autoResourcePackage qualifier="Language" />
  <autoResourcePackage qualifier="Scale" />
  <autoResourcePackage qualifier="DXFeatureLevel" />
</packaging>
```

4. Each `<autoResourcePackage>` element tells Visual Studio to automatically split the resources for the given qualifier name out into separate resource packages. This is called *auto-splitting*. With the file contents you have so far, you haven't actually changed Visual Studio's behavior. In other words, Visual Studio *already behaved as if* this file were present with these contents because these are the defaults. If you don't want Visual Studio to auto-split on a qualifier name, then delete that `<autoResourcePackage>` element from the file. Here's how the file would look if you wanted all of your language resources to be built into the app package instead of being auto-split out into separate resource packages.

XML

```
<packaging>
  <autoResourcePackage qualifier="Scale" />
  <autoResourcePackage qualifier="DXFeatureLevel" />
</packaging>
```

5. Save and close the file and rebuild your project.

To confirm that your auto-split choices are being taken into account, look for the file `<ProjectFolder>\obj\<ReleaseConfiguration folder>\split.priconfig.xml` and confirm that its contents match your choices. If they do, then you have successfully configured Visual Studio to build the resources of your choice into the app package.

There is one final step that you need to do. **But only if you deleted the `Language` qualifier name.** You need to specify the union of all of your app's supported languages as your app's default language. For details, see [Specify the default resources that your app uses](#). This is what your `priconfig.default.xml` would contain if you were including resources for English, Spanish, and French in your app package.

XML

```
<default>
  <qualifier name="Language" value="en;es;fr" />
  ...
</default>
```

How does this work?

Behind the scenes, Visual Studio launches a tool named `MakePri.exe` to generate a file known as a Package Resource Index, which describes all of your app's resources, including indicating which resource qualifier names to auto-split on. For details about this tool, see [Compile resources manually with MakePri.exe](#). Visual Studio passes a configuration file to `MakePri.exe`. The contents of your `priconfig.packaging.xml` file are used as the `<packaging>` element of that configuration file, which is the part that determines auto-splitting. So, adding and editing `priconfig.packaging.xml` ultimately influences the contents of the Package Resource Index file that Visual Studio generates for your app, as well as the contents of the packages in your app bundle.

Using a different file name than `priconfig.packaging.xml`

If you name your file `priconfig.packaging.xml`, then Visual Studio will recognize it and use it automatically. If you give it a different name, then you'll need to let Visual Studio know. In your project file, between the opening and closing tags of the first `<PropertyGroup>` element, add this XML.

XML

```
<AppxPriConfigXmlPackagingSnippetPath>FILE-PATH-AND-NAME</AppxPriConfigXmlPackagingSnippetPath>
```

Replace `FILE-PATH-AND-NAME` with the path to and name of your file.

Option 2. Use your project file to build resources into your app package

This is an alternative to Option 1. Once you understand how Option 1 works, you can choose to do Option 2 instead if that suits your development and/or build workflow better.

In your project file, between the opening and closing tags of the first `<PropertyGroup>` element, add this XML.

XML

```
<AppxBundleAutoResourcePackageQualifiers>Language|Scale|DXFeatureLevel</AppxBundleAutoResourcePackageQualifiers>
```

Here's how that looks after you've deleted the first qualifier name.

XML

```
<AppxBundleAutoResourcePackageQualifiers>Scale|DXFeatureLevel</AppxBundleAutoResourcePackageQualifiers>
```

Save and close, and rebuild your project.

There is one final step that you need to do. **But only if you deleted the `Language` qualifier name.** You need to specify the union of all of your app's supported languages as your app's default language. For details, see [Specify the default resources that your app uses](#). This is what your project file would contain if you included resources for English, Spanish, and French in your app package.

XML

```
<AppxDefaultResourceQualifiers>Language=en;es;fr</AppxDefaultResourceQualifiers>
```

Related topics

- [Package a UWP app with Visual Studio](#)
- [Compile resources manually with MakePri.exe](#)
- [Specify the default resources that your app uses](#)

Package resource indexing (PRI) APIs and custom build systems

Article • 10/20/2022

With the [package resource indexing \(PRI\) APIs](#), you can develop a custom build system for your UWP app's resources. The build system will be able to create, version, and dump (as XML) package resource index (PRI) files to whatever level of complexity your UWP app needs. If you have a custom build system that currently uses the MakePri.exe command-line tool (see [Compile resources manually with MakePri.exe](#)) then, for increased performance and control, we recommend that you switch over to calling the PRI APIs instead of calling MakePri.exe.

The PRI APIs were introduced in the Windows SDK for Windows 10, version 1803. The APIs take the form of Win32 Windows APIs, which means that you have a few options for calling them. You can call them directly from a Win32 app, or you can call them via [platform invoke](#) from a .NET app or even from a UWP app.

The scenarios in this topic demonstrate calling PRI APIs from a Win32 Visual C++ Windows Console Application project. For background info, see [Resource Management System](#).

ⓘ Note

This caveat is unlikely to be an issue, since you probably won't want to submit your custom build system app to the Microsoft Store. But, if you choose the option to develop your custom build system in the form of a UWP app, then it will be an unusual UWP app in that you won't be able to submit it to the Microsoft Store. That's because a UWP app that uses platform invoke fails Microsoft Store certification. Note that, in this case, platform invoke calls *will exist only in your custom build system; not* in your shipping UWP app (the one you're building PRI files for).

Scenario walkthroughs

Topic	Description
-------	-------------

Topic	Description
Scenario 1: Generate a PRI file from string resources and asset files	In this scenario, we'll make a new app to represent our custom build system. We'll create a resource indexer and add strings and other kinds of resources to it. Then we'll generate and dump a PRI file.

Important APIs

- [Package resource indexing \(PRI\) reference](#)

Related topics

- [Compile resources manually with MakePri.exe](#)
- [Consuming Unmanaged DLL Functions](#)
- [Resource Management System](#)

Scenario 1: Generate a PRI file from string resources and asset files

Article • 10/20/2022

In this scenario, we'll use the [package resource indexing \(PRI\) APIs](#) to make a new app to represent our custom build system. The purpose of this custom build system, remember, is to create PRI files for a target UWP app. So, as part of this walkthrough, we'll create some sample resource files (containing strings, and other kinds of resources) to represent that target UWP app's resources.

New project

Begin by creating a new project in Microsoft Visual Studio. Create a **Visual C++ Windows Console Application** project, and name it *CBSConsoleApp* (for "custom build system console app").

Choose x64 from the **Solution Platforms** drop-down.

Headers, static library, and dll

The PRI APIs are declared in the `MrmResourceIndexer.h` header file (which is installed to `%ProgramFiles(x86)%\Windows Kits\10\Include\<WindowsTargetPlatformVersion>\um\`).

Open the file `CBSConsoleApp.cpp` and include the header along with some other headers that you'll need.

C++/WinRT

```
#include <string>
#include <windows.h>
#include <MrmResourceIndexer.h>
```

The APIs are implemented in `MrmSupport.dll`, which you access by linking to the static library `MrmSupport.lib`. Open your project's **Properties**, click **Linker > Input**, edit **AdditionalDependencies** and add `MrmSupport.lib`.

Build the Solution, and then copy `MrmSupport.dll` from `C:\Program Files (x86)\Windows Kits\10\bin\<WindowsTargetPlatformVersion>\x64\` to your build output folder (probably `C:\Users\%USERNAME%\source\repos\CBSConsoleApp\x64\Debug\`).

Add the following helper function to `CBSConsoleApp.cpp`, since we'll be needing it.

C++/WinRT

```
inline void ThrowIfFailed(HRESULT hr)
{
    if (FAILED(hr))
    {
        // Set a breakpoint on this line to catch Win32 API errors.
        throw new std::exception();
    }
}
```

In the `main()` function, add calls to initialize and uninitialize COM.

C++/WinRT

```
int main()
{
    ::ThrowIfFailed(::CoInitializeEx(nullptr, COINIT_MULTITHREADED));

    // More code will go here.

    ::CoUninitialize();
}
```

Resource files belonging to the target UWP app

Now we'll need some sample resource files (containing strings, and other kinds of resources) to represent the target UWP app's resources. These, of course, can be located anywhere on your file system. But for this walkthrough it'll be convenient to put them in the project folder of `CBSConsoleApp` so that everything is in one place. You only need to add these resource files to the file system; don't add them to the `CBSConsoleApp` project.

Inside the same folder that contains `CBSConsoleApp.vcxproj`, add a new subfolder named `UWPAppProjectRootFolder`. Inside that new subfolder, create these sample resource files.

`\UWPAppProjectRootFolder\sample-image.png`

This file can contain any PNG image.

`\UWPAppProjectRootFolder\resources.resw`

XML

```
<?xml version="1.0"?>
<root>
  <data name="LocalizedString1">
    <value>LocalizedString1-neutral</value>
  </data>
  <data name="LocalizedString2">
    <value>LocalizedString2-neutral</value>
  </data>
  <data name="NeutralOnlyString">
    <value>NeutralOnlyString-neutral</value>
  </data>
</root>
```

\UWPAppProjectRootFolder\de-DE\resources.resw

XML

```
<?xml version="1.0"?>
<root>
  <data name="LocalizedString2">
    <value>LocalizedString2-de-DE</value>
  </data>
</root>
```

\UWPAppProjectRootFolder\en-US\resources.resw

XML

```
<?xml version="1.0"?>
<root>
  <data name="LocalizedString1">
    <value>LocalizedString1-en-US</value>
  </data>
  <data name="EnOnlyString">
    <value>EnOnlyString-en-US</value>
  </data>
</root>
```

Index the resources, and create a PRI file

In the `main()` function, before the call to initialize COM, declare some strings that we'll need, and also create the output folder in which we'll be generating our PRI file.

```

std::wstring projectRootFolderUWPApp{ L"UWPAppProjectRootFolder" };
std::wstring generatedPRIsFolder{ projectRootFolderUWPApp + L"\\Generated
PRIs" };
std::wstring filePathPRI{ generatedPRIsFolder + L"\\resources.pri" };
std::wstring filePathPRIDumpBasic{ generatedPRIsFolder + L"\\resources-pri-
dump-basic.xml" };

::CreateDirectory(generatedPRIsFolder.c_str(), nullptr);

```

Immediately after the call to Initialize COM, declare a resource indexer handle and then call **MrmCreateResourceIndexer** to create a resource indexer.

C++/WinRT

```

MrmResourceIndexerHandle indexer;
::ThrowIfFailed(::MrmCreateResourceIndexer(
    L"OurUWPApp",
    projectRootFolderUWPApp.c_str(),
    MrmPlatformVersion::MrmPlatformVersion_Windows10_0_0_0,
    L"language-en_scale-100_contrast-standard",
    &indexer));

```

Here's an explanation of the arguments being passed to **MrmCreateResourceIndexer**.

- The package family name of our target UWP app, which will be used as the resource map name when we later generate a PRI file from this resource indexer.
- The project root of our target UWP app. In other words, the path to our resource files. We specify this so that we can then specify paths relative to that root in subsequent API calls to the same resource indexer.
- The version of Windows that we want to target.
- A list of default resource qualifiers.
- A pointer to our resource indexer handle so that the function can set it.

The next step is to add our resources to the resource indexer that we just created.

`resources.resw` is a Resources File (.resw) that contains the neutral strings for our target UWP app. Scroll up (in this topic) if you want to see its contents. `de-DE\resources.resw` contains our German strings, and `en-US\resources.resw` our English strings. To add the string resources inside a Resources File to a resource indexer, you call **MrmIndexResourceContainerAutoQualifiers**. Thirdly, we call the **MrmIndexFile** function to a file containing a neutral image resource to the resource indexer.

C++/WinRT

```

::ThrowIfFailed(::MrmIndexResourceContainerAutoQualifiers(indexer,
    L"resources.resw"));

```

```

::ThrowIfFailed(::MrmIndexResourceContainerAutoQualifiers(indexer, L"de-
DE\\resources.resw"));
::ThrowIfFailed(::MrmIndexResourceContainerAutoQualifiers(indexer, L"en-
US\\resources.resw"));
::ThrowIfFailed(::MrmIndexFile(indexer, L"ms-resource:///Files/sample-
image.png", L"sample-image.png", L""));

```

In the call to **MrmIndexFile**, the value `L"ms-resource:///Files/sample-image.png"` is the resource uri. The first path segment is "Files", and that's what will be used as the resource map subtree name when we later generate a PRI file from this resource indexer.

Having briefed the resource indexer about our resource files, it's time to have it generate us a PRI file on disk by calling the **MrmCreateResourceFile** function.

C++/WinRT

```

::ThrowIfFailed(::MrmCreateResourceFile(indexer,
MrmPackagingModeStandaloneFile, MrmPackagingOptionsNone,
generatedPRIsFolder.c_str()));

```

At this point, a PRI file named `resources.pri` has been created inside a folder named `Generated PRIs`. Now that we're done with the resource indexer, we call **MrmDestroyIndexerAndMessages** to destroy its handle and release any machine resources that it allocated.

C++/WinRT

```

::ThrowIfFailed(::MrmDestroyIndexerAndMessages(indexer));

```

Since a PRI file is binary, it's going to be easier to view what we've just generated if we dump the binary PRI file to its XML equivalent. A call to **MrmDumpPriFile** does just that.

C++/WinRT

```

::ThrowIfFailed(::MrmDumpPriFile(filePathPRI.c_str(), nullptr,
MrmDumpType::MrmDumpType_Basic, filePathPRIDumpBasic.c_str()));

```

Here's an explanation of the arguments being passed to **MrmDumpPriFile**.

- The path to the PRI file to dump. We're not using the resource indexer in this call (we just destroyed it), so we need to specify a full file path.
- No schema file. We'll discuss what a schema is later in the topic.
- Just the basic info.

- The path of an XML file to create.

This is what the PRI file, dumped to XML here, contains.

XML

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<PriInfo>
  <ResourceMap name="OurUWPApp" version="1.0" primary="true">
    <Qualifiers>
      <Language>en-US,de-DE</Language>
    </Qualifiers>
    <ResourceMapSubtree name="Files">
      <NamedResource name="sample-image.png" uri="ms-
resource://OurUWPApp/Files/sample-image.png">
        <Candidate type="Path">
          <Value>sample-image.png</Value>
        </Candidate>
      </NamedResource>
    </ResourceMapSubtree>
    <ResourceMapSubtree name="resources">
      <NamedResource name="EnOnlyString" uri="ms-
resource://OurUWPApp/resources/EnOnlyString">
        <Candidate qualifiers="Language-en-US" isDefault="true"
type="String">
          <Value>EnOnlyString-en-US</Value>
        </Candidate>
      </NamedResource>
      <NamedResource name="LocalizedString1" uri="ms-
resource://OurUWPApp/resources/LocalizedString1">
        <Candidate qualifiers="Language-en-US" isDefault="true"
type="String">
          <Value>LocalizedString1-en-US</Value>
        </Candidate>
        <Candidate type="String">
          <Value>LocalizedString1-neutral</Value>
        </Candidate>
      </NamedResource>
      <NamedResource name="LocalizedString2" uri="ms-
resource://OurUWPApp/resources/LocalizedString2">
        <Candidate qualifiers="Language-de-DE" type="String">
          <Value>LocalizedString2-de-DE</Value>
        </Candidate>
        <Candidate type="String">
          <Value>LocalizedString2-neutral</Value>
        </Candidate>
      </NamedResource>
      <NamedResource name="NeutralOnlyString" uri="ms-
resource://OurUWPApp/resources/NeutralOnlyString">
        <Candidate type="String">
          <Value>NeutralOnlyString-neutral</Value>
        </Candidate>
      </NamedResource>
    </ResourceMapSubtree>
  </ResourceMap>
</PriInfo>
```

```
</ResourceMap>  
</PriInfo>
```

The info begins with a resource map, which is named with the package family name of our target UWP app. Enclosed by the resource map are two resource map subtrees: one for the file resources that we indexed, and another for our string resources. Notice how the package family name has been inserted into all of the resource URIs.

The first string resource is *EnOnlyString* from `en-US\resources.resw`, and it has just one candidate (which matches the *language-en-US* qualifier). Next comes *LocalizedString1* from both `resources.resw` and `en-US\resources.resw`. Consequently, it has two candidates: one matching *language-en-US*, and a fallback neutral candidate that matches any context. Similarly, *LocalizedString2* has two candidates: *language-de-DE*, and neutral. And, finally, *NeutralOnlyString* only exists in neutral form. I gave it that name to make it clear that it's not meant to be localized.

Summary

In this scenario, we showed how to use the [package resource indexing \(PRI\) APIs](#) to create a resource indexer. We added string resources and asset files to the resource indexer. Then, we used the resource indexer to generate a binary PRI file. And finally we dumped the binary PRI file in the form of XML so that we could confirm that it contains the info we expected.

Important APIs

- [Package resource indexing \(PRI\) reference](#)

Related topics

- [Package resource indexing \(PRI\) APIs and custom build systems](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Compile resources manually with MakePri.exe

Article • 04/27/2023

MakePri.exe is a command line tool that you can use to create and dump PRI files. It is integrated as part of MSBuild within Microsoft Visual Studio, but it could be useful to you for creating packages manually or with a custom build system.

ⓘ Note

MakePri.exe is installed when you check the **Windows SDK for UWP Managed Apps** option while installing the Windows Software Development Kit. It is installed to the path `%WindowsSdkDir%bin\<WindowsTargetPlatformVersion>\x64\makepri.exe` (as well as in folders named for the other architectures). For example, `C:\Program Files (x86)\Windows Kits\10\bin\10.0.17713.0\x64\makepri.exe`.

In this section

Topic	Description
MakePri.exe command-line options	MakePri.exe has the set of commands <code>createconfig</code> , <code>dump</code> , <code>new</code> , <code>resourcepack</code> , and <code>versioned</code> . This topic details the command-line options for their use.
MakePri.exe configuration file	This topic describes the schema of the MakePri.exe XML configuration file.
MakePri.exe format-specific indexers	This topic describes the format-specific indexers used by the MakePri.exe tool to generate its index of resources.

MakePri.exe command-line options

MakePri.exe has the set of commands `createconfig`, `dump`, `new`, `resourcepack`, and `versioned`. For details of their use, see [MakePri.exe command-line options](#).

MakePri.exe configuration

The PRI XML configuration file dictates how and what resources are indexed. The schema of the configuration XML is described in [MakePri.exe configuration](#).

Format-specific indexers

MakePri.exe is typically used with the `new`, `versioned`, and `resourcepack` options. In those cases it indexes source files to generate an index of resources. MakePri.exe uses various individual indexers to read different source resource files or containers for resources. The simplest indexer is the folder indexer, which indexes the contents of a folder for resources such as `.jpg` or `.png` images. For more info, see [MakePri.exe format-specific indexers](#).

MakePri.exe warnings and error messages

Resources found for language(s) '<language(s)>' but no resources found for default language(s): '<language(s)>'. Change the default language or qualify resources with the default language.

This warning is displayed when MakePri.exe or MSBuild discovers files or string resources for a given named resource that appear to be marked with language qualifiers, but no candidate is found for a default language. The process for using qualifiers in file and folder names is described in [Tailor your resources for language, scale, and other qualifiers](#). A file or folder may have a language name in it, but no resources are discovered that are qualified for the exact default language. For example, if a project uses "en-US" as the default language and has a file named "de/logo.png", but does not have any files that are marked with the default language "en-US", this warning will appear. In order to remove this warning, either file(s) or string resource(s) should be qualified with the default language, or the default language should be changed. To change the default language, with your solution open in Visual Studio, open `Package.appxmanifest`. On the Application tab, confirm that the Default language is set appropriately (for example, "en" or "en-US").

No default or neutral resource is given for '<resource identifier>'. The application may throw an exception for certain user configurations when retrieving the resources.

This warning is displayed when MakePri.exe or MSBuild discovers files or resources that appear to be marked with language qualifiers for which the resources are unclear. There are qualifiers, but there is no guarantee that a particular resource candidate can be returned for that resource identifier at run time. If no resource candidate for a particular language, home region, or another qualifier can be found that is default or will always match the context of a user, this warning will be displayed. At run time, for particular user configurations such as a user's language preferences or home location (**Settings > Time & Language > Region & language**), the APIs used to retrieve the resource may throw an unexpected exception. In order to remove this warning, default resources should be provided, such as a resource in the project's default language or global home region (homeregion-001).

Using MakePri.exe in a build system

Build systems should use the MakePri.exe `new`, `versioned`, or `resourcepack` command, depending on the type of project being built. Build systems that create a fresh PRI file should use the `new` command. Build systems that must ensure compatibility of internal offsets through iterations can use the `versioned` command. Build systems that must create a PRI file that contains additional variants of resources, with validation to ensure that no new resources are added for that variant, should use the `resourcepack` command.

Build systems that require explicit control over source files that get indexed can use the ResFiles indexer instead of indexing a folder. Build systems can also use multiple index passes with different [format-specific indexers](#) to generate a single PRI file.

Build systems can also use the PRI format-specific indexer to add pre-built PRI files into the PRI for the package from other components, such as class libraries, assemblies, SDKs, and DLLs.

When PRI files are built for other components, class libraries, assemblies, DLLs, and SDKs, the `initialPath` configuration should be used to ensure component resources have their own sub-resource maps that don't conflict with the app they're included in.

Related topics

- [MakePri.exe command-line options](#)
- [MakePri.exe configuration](#)
- [MakePri.exe format-specific indexers](#)
- [Tailor your resources for language, scale, and other qualifiers](#)

MakePri.exe command-line options

Article • 10/20/2022

[MakePri.exe](#) has the set of commands `createconfig`, `dump`, `new`, `resourcepack`, and `versioned`. This topic details the command-line options for their use.

ⓘ Note

MakePri.exe is installed when you check the **Windows SDK for UWP Managed Apps** option while installing the Windows Software Development Kit. It is installed to the path `%WindowsSdkDir%bin\<WindowsTargetPlatformVersion>\x64\makepri.exe` (as well as in folders named for the other architectures). For example, `C:\Program Files (x86)\Windows Kits\10\bin\10.0.17713.0\x64\makepri.exe`.

Getting help from the command-line

You can run `MakePri.exe help` or `MakePri.exe /?` to see the commands that you can use with MakePri.exe. You can also issue `MakePri.exe <command> /?` to see specifics about a command and, in very rare cases, even `MakePri.exe <command> <option>` to see specifics about an option.

MakePri commands

Console

```
C:\>makepri help
```

Usage:

```
MakePri.exe <command> [options]
```

Example:

```
MakePri.exe new /cf C:\MyApp\priconfig.xml /pr C:\MyApp\src\ /in  
PackageName
```

Description:

Creates, dumps, and performs utility functions on a PRI file. A PRI file is an index of application resources, such as strings and image files.

Command Options:

MakePri.exe createconfig	Creates a PRI config file for use with other commands
MakePri.exe dump	Dumps the contents of a PRI file
MakePri.exe new	Creates a new PRI file from scratch
MakePri.exe resourcepack	Creates a PRI file that contains additional resource variants for a base PRI file
MakePri.exe versioned	Creates a PRI file based on a previous version

Help:

MakePri.exe help	Show this help page
MakePri.exe <command> /?	Shows detailed help for <command>

For example,
MakePri.exe createconfig /?

Createconfig command

The `createconfig` command creates a new, initialized PRI config file defining the qualifier defaults that you specify. Run `MakePri.exe createconfig /?` to see detailed help for this command.

Console

```
C:\>makepri createconfig /?
```

Usage:

```
MakePri.exe createconfig /cf <config file destination> /dq  
<default qualifiers> [options]
```

Example:

```
MakePri.exe createconfig /cf C:\MyApp\priconfig.xml /dq lang-en-US /o  
/pv 10.0.0
```

Description:

Creates a PRI configuration file at <config file destination> with default qualifiers specified by <default qualifiers>. Multiple qualifiers are separated by underscores (_)

Required Parameters:

/ConfigXml(cf)	: <FILEPATH> Configuration file output location
/Default(dq)	: <QUALIFIERS> The default qualifiers to set in the

configuration file. A language qualifier is required

Options:

/ExtensionDll(ex) : <FILEPATH> Location of the Resource Management System environment extension DLL. This DLL must be signed by a Microsoft-issued certificate. Default is an empty path (no DLL will be used)

/Overwrite(o) : Overwrite an existing output file of the same name without prompting

/Platform(pv) : <VERSION> Platform version to use for generated configuration file

FILEPATH - a path to a file, either relative to the current directory or absolute

QUALIFIERS - a valid qualifier token (for example, lang-en-US_scale-100_contrast-high)

Help:

/Help(h, ?) : Display the usage help text

Dump command

The `dump` command outputs a dumped xml file containing a list of all resources in a specified PRI file. Run `MakePri.exe dump /?` to see detailed help for this command.

ⓘ Note

A schema-free resource pack is one that was created with the `omitSchemaFromResourcePacks` switch in the PRI config file. To dump a schema-free resource pack, use the switch `/es <main_package_PRI_file>`. If you don't specify the main file then you'll see the error message *"The resources.pri in the package was corrupted so encryption failed (error PRI222: 0xdef0000f - Unspecified error occurred)"*.

Console

```
C:\>makepri dump /?
```

Usage:

```
MakePri.exe dump [options]
```

Example:

```
MakePri.exe dump /if C:\MyApp\resources.pri /of C:\resources.pri.xml
```

Description:

Outputs a dumped xml file at <output file> containing a list of all resources in <index file>.

Options:

/DumpType(dt)	: <STRING> Format of the dumped file, default is Basic
/ExtensionDll(ex)	: <FILEPATH> Location of the Resource Management System environment extension DLL. This DLL must be signed by a Microsoft-issued certificate. Default is an empty path
/ExternalSchema(es)	: <FILEPATH> Location of the external schema file (no DLL will be used)
/IndexFile(if)	: <FILEPATH> Location of the PRI file to dump from. Default is .\resources.pri
/OutputFile(of)	: <FILEPATH> Output location of the dump file, default is .\[indexfile].xml
/OutputOptions(oo)	: <OPTIONS> Options to provide detailed control over contents of XML output files.
/Overwrite(o)	: Overwrite an existing output file of the same name without prompting
/Verbose(v)	: Causes verbose messages to be output to the console

Dump Type:

Either 'Basic', 'Detailed', 'Schema', or 'Summary'

FILEPATH	- a path to a file, either relative to the current directory or absolute
----------	--

Help:

/Help(h, ?)	: Display the usage help text
-------------	-------------------------------

New command

The `new` command creates a new PRI file by indexing the files in your project as directed by your configuration file. Run `MakePri.exe new /?` to see detailed help for this command.

```
C:\>makepri new /?
```

Usage:

```
MakePri.exe new /cf <config file> /pr <project root> [options]
```

Example:

```
MakePri.exe new /cf C:\MyApp\priconfig.xml /pr C:\MyApp\src\  
/mn C:\MyApp\AppXManifest.xml /o /of C:\MyApp\src\resources.pri
```

Description:

Creates a PRI file at <output file> by indexing all files in <project root> and its subdirectories as directed by <config file>. The index will be assigned <index name> to reference resources in the app

Required Parameters:

```
/ConfigXml(cf) : <FILEPATH> Configuration file location. Use the  
                'Makepri.exe createconfig' command to generate one  
/ProjectRoot(pr) : <FOLDERPATH> Root location of project files
```

Options:

```
/AutoMerge(am) : This flag is not recommended for normal use with  
.appx  
                packages. It causes Makepri.exe to set the auto  
                merge flag within the PRI file. Default is not set.  
/ExtensionDll(ex) : <FILEPATH> Location of the Resource Management  
System  
                environment extension DLL. This DLL must be signed  
by  
                a Microsoft-issued certificate. Default is an empty  
path  
                (no DLL will be used)  
/IndexLog(il) : <FILEPATH> XML Log of indexed resources, no file  
                generated by default  
/IndexName(in) : <STRING> Name for the generated index of resources.  
                Typically matches the .appx package name, class  
library  
                simple name, etc. May be supplied via the  
                [manifest] parameter.  
/IndexOptions(io) : <OPTIONS> Options to provide detailed control over  
                behavior of resource indexers.  
/Manifest(mn) : <FILEPATH> Location of the application or  
component's  
                manifest. This parameter is ignored if [indexname]  
                is given. Default is [projectroot]\AppXManifest.xml  
/MappingFile(mf) : <MAPPINGFILETYPE> Generate a mapping file in the  
given  
                file format.  
/OutputFile(of) : <FILEPATH> Output location of PRI file, default is  
                .\resources.pri
```

```

/Overwrite(o)      : Overwrite an existing output file of the same name
                    without prompting
/ReverseMap(rm)    : Generate a reverse mapping section in the PRI file
                    which can be used for debugging purposes.
/SchemaFile(sf)    : <FILEPATH> Output location of XML resource schema
                    description.
/Verbose(v)        : Causes verbose messages to be output to the console
/VersionMajor(vma): <INTEGER> [Deprecated] Major version number for
                    index, default is 1

```

```

FOLDERPATH        - a path to a folder
FILEPATH           - a path to a file, either relative to the current
                    directory or absolute
MAPPINGFILETYPE    - Supported File type(s): 'AppX'

```

Help:

```

/Help(h, ?)       : Display the usage help text

```

ResourcePack command

The `resourcepack` command creates a new PRI file by indexing the files in your project as directed by your configuration file. A resource pack PRI file contains only additional variants of resources already specified in an existing PRI file. Run `MakePri.exe resourcepack /?` to see detailed help for this command.

Console

```

C:\>makepri resourcepack /?

```

Usage:

```

MakePri.exe resourcepack /pr <project root> /cf <config file> [options]

```

Example:

```

MakePri.exe resourcepack /cf C:\MyAppEs\priconfig.xml /pr
C:\MyAppEs\src\
/if C:\MyApp\1.2\Resources.pri /o /of C:\MyAppEs\Resources.pri

```

Description:

Creates a PRI file at <output file> by indexing all files in <project root> and its subdirectories as directed by <config file>. A resource pack PRI file contains only additional variants of resources already specified in <index file>.

Required Parameters:

```

/ConfigXml(cf)     : <FILEPATH> Configuration file location. Use

```

```

        'Makepri.exe createconfig' command to generate one
/ProjectRoot(pr) : <FOLDERPATH> Root location of project files

Options:
-----
    /AutoMerge(am) : This flag is not recommended for normal use with
.appx
                    packages. It causes Makepri.exe to set the auto
                    merge flag within the PRI file. By default it is set
                    to same as the base PRI file.
    /ExtensionDll(ex) : <FILEPATH> Location of the Resource Management
System
                    environment extension DLL. This DLL must be signed
by
                    a Microsoft-issued certificate. Default is an empty
path
                    (no DLL will be used)
    /IndexFile(if) : <FILEPATH> Location of the base PRI or XML schema
file.
                    Default is <ProjectRoot>\resources.pri
    /IndexLog(il) : <FILEPATH> XML Log of indexed resources, no file
                    generated by default
    /IndexOptions(io) : <OPTIONS> Options to provide detailed control over
                    behavior of resource indexers.
    /MappingFile(mf) : <MAPPINGFILETYPE> Generate a mapping file in the
given
                    file format.
    /OutputFile(of) : <FILEPATH> Output location of PRI file, default is
                    .\resources.pri
    /Overwrite(o) : Overwrite an existing output file of the same name
                    without prompting
    /ReverseMap(rm) : Generate a reverse mapping section in the PRI file
                    which can be used for debugging purposes.
    /SchemaFile(sf) : <FILEPATH> Output location of XML resource schema
                    description.
    /Verbose(v) : Causes verbose messages to be output to the console

FOLDERPATH - a path to a folder
FILEPATH - a path to a file, either relative to the current
           directory or absolute
MAPPINGFILETYPE - Supported File type(s): 'AppX'

Help:
-----
    /Help(h, ?) : Display the usage help text

```

Versioned command

The `versioned` command creates a versioned PRI file by indexing the files in your project as directed by your configuration file. Run `MakePri.exe versioned /?` to see detailed help for this command.

```
C:\>makepri versioned /?
```

Usage:

```
-----
```

```
MakePri.exe versioned /cf <config file> /pr <project root> [options]
```

Example:

```
-----
```

```
MakePri.exe versioned /cf C:\MyApp\priconfig.xml /pr C:\MyApp\src
/if C:\MyApp\1.2\resources.pri /o /of C:\MyApp\src\resources.pri /o
```

Description:

```
-----
```

Creates a versioned PRI file at <output file> by indexing all files in <project root> and its subdirectories as directed by <config file>.

Required Parameters:

```
-----
```

```
/ConfigXml(cf)      : <FILEPATH> Configuration file location. Use
                      'Makepri.exe createconfig' command to generate one
/ProjectRoot(pr)    : <FOLDERPATH> Root location of project files
```

Options:

```
-----
```

```
/AutoMerge(am)      : This flag is not recommended for normal use with
                      .appx
                      packages. It causes Makepri.exe to set the auto
                      merge flag within the PRI file. By default it is set
                      to same as the base PRI file.
/ExtensionDll(ex)    : <FILEPATH> Location of the Resource Management
System
                      environment extension DLL. This DLL must be signed
by
                      a Microsoft-issued certificate. Default is an empty
path
                      (no DLL will be used)
/IndexFile(if)       : <FILEPATH> Location of the base PRI or XML schema
file
                      to version from. Default is
<ProjectRoot>\resources.pri
/IndexLog(il)         : <FILEPATH> XML Log of indexed resources, no file
                      generated by default
/IndexOptions(io)     : <OPTIONS> Options to provide detailed control over
                      behavior of resource indexers.
/MappingFile(mf)     : <MAPPINGFILETYPE> Generate a mapping file in the
given
                      file format.
/OutputFile(of)       : <FILEPATH> Output location of PRI file, default is
                      [current directory]\resources.pri
/Overwrite(o)         : Overwrite an existing output file of the same name
                      without prompting
/ReverseMap(rm)       : Generate a reverse mapping section in the PRI file
```

```

which can be used for debugging purposes.
/SchemaFile(sf)      : <FILEPATH> Output location of XML resource schema
                      description.
/Verbose(v)          : Causes verbose messages to be output to the console

FOLDERPATH           - a path to a folder
FILEPATH              - a path to a file, either relative to the current
                      directory or absolute
MAPPINGFILETYPE       - Supported File type(s): 'AppX'

Help:
-----
/Help(h, ?)          : Display the usage help text

```

/ExtensionDll(ex)

You use the extension DLL option (/ex) with `createconfig`, `dump`, `new`, `resourcepack`, and `versioned` to specify the location of the Resource Management System environment extension DLL.

Logging/metadata file

MakePri can include info specific to a resource pack in the indexer metadata file. Here is an example of a log file for `resources.pri` with resource PRI files `german.pri` and `highresolution.pri`.

XML

```

<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<root>
  <package filename="resources.pri">
    <instance itemname="Files\logo.jpg" qualifiers="Scale-100" src=""
type="Path">
      <value>logo.scale-100.jpg</value>
    </instance>
    <instance itemname="resources\string2" qualifiers="Language-en-us"
src="C:\Users\alias\Desktop\repro\app4\project\en-us\resources.resw"
type="String">
      <value>value2</value>
    </instance>
    <instance itemname="resources\string1" qualifiers="Language-en-us"
src="C:\Users\alias\Desktop\repro\app4\project\en-us\resources.resw"
type="String">
      <value>value1</value>
    </instance>
  </package>
  <package filename="german.pri">
    <instance itemname="resources\string2" qualifiers="Language-de-de"

```

```

src="C:\Users\alias\Desktop\repro\app4\project\de-de\resources.resw"
type="String">
  <value>value2</value>
</instance>
<instance itemname="resources\string1" qualifiers="Language-de-de"
src="C:\Users\alias\Desktop\repro\app4\project\de-de\resources.resw"
type="String">
  <value>value1</value>
</instance>
</package>
<package filename="highresolution.pri">
  <instance itemname="Files\logo.jpg" qualifiers="Scale-200" src=""
type="Path">
  <value>logo.scale-200.jpg</value>
</instance>
</package>
</root>

```

/IndexFile(if) option

You use the index file option (/if) with `dump`, `resourcepack`, and `versioned` to specify an input PRI file.

For `resourcepack` and `versioned`, instead of providing a PRI file as the input parameter for /IndexFile(if), you can instead provide a schema file.

Console

```
/IndexFile(if) <FILEPATH>
```

FILEPATH is a token that specifies the location of the input PRI file or PRI schema file.

/IndexOptions(io) option

You use the index options option (/io) with `new`, `resourcepack`, and `versioned` to specify options that provide detailed control over the behavior of resource indexers. Index options are disabled by default.

Console

```
/IndexOptions(io) <OPTIONS>
```

OPTIONS is a comma-separated list comprised of the following options.

- +/-HiddenFiles(hf). Index (+) or ignore (-) hidden files and folders.
- +/-LinkedFiles(lf). Index (+) or ignore (-) linked files and folders.

/MappingFile(mf) option

You use the mapping file option (/mf) with `new`, `resourcepack`, and `versioned` to generate a mapping file. [MakeAppx.exe](#) uses the mapping file to generate app packages.

Console

```
/MappingFile(mf) <MAPPINGFILETYPE>
```

MAPPINGFILETYPE is a token that specifies the format of the mapping file. The only valid supported format is `appx`.

Console

```
/mf appx
```

This is an example contents of a main mapping file.

Console

```
"ResourceDimensions"           "language-de-de"
```

And this is an example contents of a resource pack mapping file.

Console

```
"ResourceId"                   "Resources184.1a5decaf08"
"ResourceDimensions"           "language-de-de"
```

Output summary

If resource packs are created, the output summary from MakePRI.exe is of more verbose form. Here's an example.

Console

```
Index Pass Completed: ResourcePackTests\TestApp_ResourcePack
Language Qualifiers: fr-FR, de-DE
```

```
Finished building
Version: 1.0
Resource Map Name: AppTest
Named Resources: 11

Resource PRI: fr-FR.pri
Version: 1.0
Resource Candidates: 4
Language: fr-FR

Resource PRI: de-DE.pri
Version: 1.0
Resource Candidates: 4
Language: de-DE

Output File(s) at TempTestResults
Successfully Completed
```

/Overwrite(o) option

If the over-write option (/o) is not provided, and the specified output file(s) already exist(s), then MakePri.exe requires a confirmation before overwriting.

Console

```
Following file(s) already exist at output location:
<file(s)>
Overwrite these file(s)? [Y]es (any other key to cancel):
```

/OutputFile(of) option

You use the output file option (/of) with `dump`, `new`, `resourcepack`, and `versioned` to specify the output location and name of the PRI file to be generated. If MakePri.exe generates more than one resource PRI file, it places them in the parent folder of the target file. For example, if you specify `/of MyParentFolder\TargetFile.pri` then MakePri.exe generates `TargetFile.language-en.pri` and `TargetFile.scale-100.pri` alongside `TargetFile.pri` under `ParentFolder`.

Here is an example error condition and the corresponding error message.

Error condition	Error message
-----------------	---------------

Error condition	Error message
The output file name is the same as one of the resource pack names in the configuration.	Invalid Configuration: Resource Pack name <resource pack name> cannot be the same as the output file <outputfilename.pri>.

/ReverseMap(rm) option

You use the reverse map option (/rm) with `new`, `resourcepack`, and `versioned` to generate a reverse-mapping section in the PRI file, which can be used for debugging.

/SchemaFile(sf) option

You use the schema file option (/sf) with `new`, `resourcepack`, and `versioned` to write a schema file at the specified location.

For `resourcepack` and `versioned`, instead of providing a PRI file as the input parameter for /IndexFile(if), you can instead provide a schema file.

Console

```
/SchemaFile(sf) <FILEPATH>
```

FILEPATH is a token that specifies where to write the schema file.

This is an example of a schema file.

XML

```
<PriInfo>
  <ResourceMap name="IndexName" resourceVersion="1.0">
    <ResourceMapSubtree name="Resources" index="1">
      <NamedResource name="String1" index="1"/>
      <NamedResource name="String2" index="1"/>
    </ResourceMapSubtree>
    <ResourceMapSubtree name="Files" index="2">
      <NamedResource name="logo.png" index="2"/>
      <ResourceMapSubtree name="images" index="3">
        <NamedResource name="success.png" index="3"/>
        <NamedResource name="error.png" index="3"/>
      </ResourceMapSubtree>
    </ResourceMapSubtree>
  </ResourceMap>
</PriInfo>
```

/VersionMajor(vma) is deprecated

The major version (/vma) option (for the `new` command) is deprecated, and using it results in this warning message.

Console

```
'VersionMajor (vma)' input parameter has been deprecated. Please specify  
major version in the configuration file using 'majorVersion' attribute on  
'resources' node.
```

To provide the major version number, use the [resources@majorVersion](#) attribute in your configuration file.

Related topics

- [MakePri.exe](#)

MakePri.exe configuration file

Article • 10/20/2022

This topic describes the schema of the [MakePri.exe](#) XML configuration file; also known as a PRI config file. The MakePri.exe tool has a [createconfig command](#) that you can use to create a new, initialized PRI config file.

ⓘ Note

MakePri.exe is installed when you check the **Windows SDK for UWP Managed Apps** option while installing the Windows Software Development Kit. It is installed to the path `%WindowsSdkDir%bin\<WindowsTargetPlatformVersion>\x64\makepri.exe` (as well as in folders named for the other architectures). For example, `C:\Program Files (x86)\Windows Kits\10\bin\10.0.17713.0\x64\makepri.exe`.

The PRI config file controls what resources are indexed, and how. The configuration XML must conform to the following schema.

XML

```
<?xml version="1.0" encoding="utf-8"?>
<xs:schema attributeFormDefault="unqualified" elementFormDefault="qualified"
  xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="resources">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="packaging" maxOccurs="1" minOccurs="0">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="autoResourcePackage" maxOccurs="unbounded"
minOccurs="0">
                <xs:complexType>
                  <xs:attribute name="qualifier" type="xs:string"
use="required" />
                </xs:complexType>
              </xs:element>
              <xs:element name="resourcePackage" maxOccurs="unbounded"
minOccurs="0">
                <xs:complexType>
                  <xs:sequence>
                    <xs:element name="qualifierSet" maxOccurs="unbounded"
minOccurs="0">
                      <xs:complexType>
                        <xs:attribute name="definition" type="xs:string"
use="required" />
                      </xs:complexType>
                    </xs:element>
                  </xs:sequence>
                </xs:complexType>
              </xs:element>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```

        </xs:sequence>
        <xs:attribute name="name" type="xs:string" use="required"
/>

    </xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
<xs:element maxOccurs="unbounded" name="index">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="qualifiers" minOccurs="0"
maxOccurs="unbounded">
                <xs:complexType>
                    <xs:sequence>
                        <xs:element minOccurs="1" maxOccurs="unbounded"
name="qualifier">
                                <xs:complexType>
                                    <xs:attribute name="name" type="xs:string"
use="required" />
                                    <xs:attribute name="value" type="xs:string"
use="required" />
                                </xs:complexType>
                            </xs:element>
                        </xs:sequence>
                    </xs:complexType>
                </xs:element>
            <xs:element name="default" minOccurs="0"
maxOccurs="unbounded">
                <xs:complexType>
                    <xs:sequence>
                        <xs:element minOccurs="1" maxOccurs="unbounded"
name="qualifier">
                                <xs:complexType>
                                    <xs:attribute name="name" type="xs:string"
use="required" />
                                    <xs:attribute name="value" type="xs:string"
use="required" />
                                </xs:complexType>
                            </xs:element>
                        </xs:sequence>
                    </xs:complexType>
                </xs:element>
            <xs:element name="indexer-config" minOccurs="0"
maxOccurs="unbounded">
                <xs:complexType>
                    <xs:sequence>
                        <xs:any minOccurs="0" maxOccurs="unbounded"
processContents="skip"/>
                    </xs:sequence>
                    <xs:attribute name="type" type="xs:string" use="required"
/>
                </xs:complexType>
            </xs:element>
        </xs:sequence>
    </xs:complexType>
</xs:element>

```

```

        </xs:sequence>
        <xs:attribute name="root" type="xs:string" use="required" />
        <xs:attribute name="startIndexAt" type="xs:string"
use="required" />
    </xs:complexType>
</xs:element>
</xs:sequence>
<xs:attribute name="isDeploymentMergeable" type="xs:boolean"
use="optional" />
    <xs:attribute name="majorVersion" type="xs:positiveInteger"
use="optional" />
    <xs:attribute name="targetOsVersion" type="xs:string" use="optional"
/>
</xs:complexType>
</xs:element>
</xs:schema>

```

- The `default` element specifies the context (language, scale, contrast, etc.) that should be used to resolve resources when the runtime context does not match any resource candidates. Because this context is specified at build time and does not change, resources are resolved to this context as qualifiers are created. The matched score is stored at build time. Every qualifier must have some value specified. See [ResourceContext](#) for details on how resources are chosen.
- The `index` element defines discrete indexing passes that are done over the assets. Each indexing pass determines the [format-specific indexers](#) to use, and which resources to index.
- The `qualifiers` element sets the initial qualifiers for the first file or folder that other resources inherit. Each qualifier element must have a valid name and value (see [Tailor your resources for language, scale, high contrast, and other qualifiers](#)).
- The `root` attribute is the path root of the physical file for the index pass. It can be relative or absolute. If relative, it is appended to the project root that you provide in the command line. If absolute, it is directly used as the index pass root. Back or forward slashes are acceptable. Trailing slashes are trimmed. The root of the index pass determines the folder to which all resources are considered relative.
- The `startIndexAt` attribute is the initial seed file or folder used in indexing. It is relative to the index pass root. An empty value assumes the index pass root folder.

Default PRI config file

MakePri.exe generates this new, initialized PRI config file when the [createconfig command](#) is issued.

```

<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<resources targetOsVersion="10.0.0" majorVersion="1">
  <packaging>
    <autoResourcePackage qualifier="Language"/>
    <autoResourcePackage qualifier="Scale"/>
    <autoResourcePackage qualifier="DXFeatureLevel"/>
  </packaging>
  <index root="\\" startIndexAt="\\">
    <default>
      <qualifier name="Language" value="en-US"/>
      <qualifier name="Contrast" value="standard"/>
      <qualifier name="Scale" value="100"/>
      <qualifier name="HomeRegion" value="001"/>
      <qualifier name="TargetSize" value="256"/>
      <qualifier name="LayoutDirection" value="LTR"/>
      <qualifier name="Theme" value="dark"/>
      <qualifier name="AlternateForm" value=""/>
      <qualifier name="DXFeatureLevel" value="DX9"/>
      <qualifier name="Configuration" value=""/>
      <qualifier name="DeviceFamily" value="Universal"/>
      <qualifier name="Custom" value=""/>
    </default>
    <indexer-config type="folder" foldernameAsQualifier="true"
filenameAsQualifier="true" qualifierDelimiter="."/>
    <indexer-config type="resw" convertDotsToSlashes="true" initialPath=""/>
    <indexer-config type="resjson" initialPath=""/>
    <indexer-config type="PRI"/>
  </index>
  <!--<index startIndexAt="Start Index Here" root="Root Here">-->
  <!--      <indexer-config type="resfiles" qualifierDelimiter="."/>-->
  <!--      <indexer-config type="priinfo" emitStrings="true"
emitPaths="true" emitEmbeddedData="true"/>-->
  <!--</index>-->
</resources>

```

Packaging element

The `packaging` element defines PRI split information. The schema for the `packaging` element is defined for both automatic (support for `autoResourcePackage` along a specific dimension), and manual configuration.

This example shows how to use `autoResourcePackage` along a specific dimension.

XML

```

<packaging>
  <autoResourcePackage qualifier="Language"/>
  <autoResourcePackage qualifier="Scale"/>

```